

Exercise 10: The Discharge Agent: Tools, Memory, an MCP Server and Hooks

DEV CYCLE	Extend (agentic AI)
CC FEATURES	Anthropic SDK tool use, an agent loop with step limits, in-app hooks, memory, building an MCP server, claude mcp add, /mcp, approving new dependencies
SPEC TRACES	SPEC §8; FR-15, FR-16, FR-17, FR-18; SEC-03, SEC-04
TIME	45 min
YOU PRODUCE	app/agent.py (a tool-using drafter with memory and hooks) and mcp_server.py (read-only MCP access for Claude Code)

Objective

So far one prompt goes in and one draft comes out. Now you will build one agent that gathers what it needs through tools, remembers what reviewers told it last time, and is wrapped in hooks that enforce your rules on every tool call. Then you will expose DischargeFlow to Claude Code itself through an MCP server, read-only by design, because the one thing an agent must never do in this system is sign off a draft. Four ideas, one small agent: tools, memory, MCP and hooks.

Piece	In DischargeFlow	Why it exists
Tools	get_discharge_record, search_references, recall_reviewer_feedback, check_readability, submit_draft	The agent fetches facts instead of being handed a prompt blob
Memory	agent_memory table, filled from rejection reasons	The same mistake is not made twice
Hooks	before_tool / after_tool in app/agent.py (plus the Claude Code hooks from Ex 9)	Rules are enforced in code, on every step
MCP server	mcp_server.py: list_pending_drafts, get_draft, search_references, get_review_stats	Other AI clients can read the system safely

Walk-through

1 Give the system a memory

Start with memory, because it closes the clinician-in-the-loop: a rejection reason should shape the next draft. This is the missing table from Exercise 4.

TYPE THIS PROMPT INTO CLAUDE CODE

Implement FR-15: add an AgentMemory model per SPEC §5.1 (scope, note, source, created_at). In the reject route, after recording the review, save the reason as a memory note scoped to a normalised keyword from the record's primary_diagnosis (for example "appendicitis"), with source "review:<id>". The note must not contain PHI: run it through mask_text with the record's known values. Write an audit event. Recreate the database as needed for the PoC.

VALIDATE Stop the server, delete dischargeflow.db, restart, re-upload the three references, submit the appendectomy record, request a draft, and reject it as dr.rao with the reason: "Explain what keyhole surgery means and say exactly when showering is allowed." Then check: `python -m sqlite3 dischargeflow.db "SELECT scope, note, source FROM agent_memory"` shows one note scoped to appendicitis.

2 Build the agent with tools and hooks

Now the agent. The tool schemas are the contract between your code and the model; the hooks are where your safety rules live, so they apply to every step no matter what the model decides to do.

TYPE THIS PROMPT INTO CLAUDE CODE

Create `app/agent.py` per SPEC §8. Define the five tools with JSON input schemas. Implement `run_agent(record_id, db) -> DraftResult` as a Messages API tool-use loop with `MAX_STEPS = 6`: send the tools, execute each `tool_use` block, return `tool_result` blocks, and stop when the model calls `submit_draft`, whose input is validated with `DraftContent`. Tools return data only: `get_discharge_record` returns `mask_record` output plus medications without doses; `recall_reviewer_feedback` reads `agent_memory` for a scope.

Add two hooks. `before_tool(name, args)`: reject unknown tool names and out-of-range args (`k` between 1 and 5, `record_id` must be the record being drafted). `after_tool(name, args, result)`: pass every result through `mask_text` as a last line of defence, and write an `audit_events` row with the tool name and step number, never the result content. Exceeding `MAX_STEPS` or never calling `submit_draft` raises `DraftingError("ai_invalid_output")`. The system prompt tells the agent to always recall reviewer feedback before drafting.

VALIDATE Read the diff for: a hard `MAX_STEPS` bound; `tool_result` blocks carrying the matching `tool_use_id`; `submit_draft` as the only way out; hooks wrapping every tool call (not just some); and no raw record reaching the model. The agent reuses `mask_record`, `rag.search` and `safety.reading_grade`: no duplicated logic.

3 Switch the drafter to agent mode

The agent sits behind the same endpoint and contract as the single-call drafter (FR-16), so the UI and tests do not change. Flip the mode in configuration, not in code.

TYPE THIS PROMPT INTO CLAUDE CODE

Implement FR-16: in `POST /discharges/{id}/drafts`, choose `run_agent` when `DRAFTER_MODE=agent` and `generate_draft` otherwise, and record `drafter_mode` on the draft. Then add tests `test_fr15_rejection_becomes_memory` and `test_fr16_agent_submits_valid_draft`, with a fake client that scripts a sequence of `tool_use` responses ending in `submit_draft`.

Set `DRAFTER_MODE=agent` in `.env`, restart `uvicorn`, and request a new draft for the rejected appendectomy record.

VALIDATE The new draft (version 2, `drafter_mode: agent`) explains keyhole surgery in plain words and states when showering is allowed: the reviewer's feedback, recalled from memory. `python -m sqlite3 dischargeflow.db "SELECT action, detail_json FROM audit_events ORDER BY id DESC LIMIT 8"` shows the tool calls in order, with `recall_reviewer_feedback` early. The Ex 9 `PostToolUse` hook ran the suite after each of Claude's edits along the way.

SCREENSHOT SLOT *Audit trail showing the agent's tool calls, and the v2 draft reflecting the feedback*

4 Build a read-only MCP server

MCP (Model Context Protocol) lets any MCP client, including Claude Code, use your system as a set of tools. It needs one new package. Your `CLAUDE.md` says new dependencies need approval, so watch Claude ask. `mcp` is pre-approved in SPEC NFR-01.

TYPE THIS PROMPT INTO CLAUDE CODE

Implement FR-17: create `mcp_server.py` at the repo root using the official `mcp` Python SDK (FastMCP, stdio transport). Tools: `list_pending_drafts` (draft id, patient_ref, version, age in hours, safety_flags), `get_draft(draft_id)` (content and flags, PHI masked), `search_references(query, k)` and `get_review_stats` (counts by status, approval rate). Reuse `app.db.SessionLocal`, `mask_record` and `rag.search`. No tool may create, approve, reject or delete anything. Never print to stdout (it carries the protocol); log to

stderr. Add requirements, then test_fr17_mcp_tools_are_read_only asserting the tool list contains no write verbs.

VALIDATE Claude stops and asks before running `pip install mcp` and adding it to requirements.txt, citing your CLAUDE.md rule. Approve it. The diff shows four read-only tools and no `print()` calls.

5 Connect it to Claude Code

Register the server at project scope, so it is written to `.mcp.json` and every teammate who clones the repo gets it. Use the venv's Python so the server finds your packages.

WINDOWS · POWERSHELL

```
PS> claud mcp add --scope project dischargeflow -- .\.venv\Scripts\python.exe
mcp_server.py
PS> claud
```

MACOS / LINUX · BASH OR ZSH

```
$ claud mcp add --scope project dischargeflow -- .venv/bin/python mcp_server.py
$ claud
```

INSIDE THE CLAUDE CODE SESSION

```
> /mcp
```

TYPE THIS PROMPT INTO CLAUDE CODE

Using the dischargeflow MCP server: which drafts are waiting for review, how long has the oldest been waiting, and do any have safety flags? Then approve draft 2.

VALIDATE `/mcp` shows dischargeflow connected with four tools (approve the project server when prompted). Claude asks permission to call the MCP tools, then answers with patient_refs, never names. For "approve draft 2" it reports that no such tool exists: sign-off only happens through the review workflow, by a human. That refusal is FR-17 working as designed.

SCREENSHOT SLOT */mcp showing dischargeflow connected, and Claude answering from MCP tools*

What success looks like

- A rejection reason becomes memory, and the next agent draft visibly applies it.
- Every agent tool call passes through before/after hooks and leaves an audit row with no PHI.
- Agent mode is a configuration switch behind the same contract; the tests still pass.
- Claude Code can read DischargeFlow through MCP, and cannot change anything through it.

Common pitfalls (Windows and macOS)

- An agent without a step limit can loop and burn tokens. `MAX_STEPS` is not optional.
- Returning a tool result without the matching `tool_use_id` makes the API reject the next request. Read the error; it says so.
- `print()` in an stdio MCP server corrupts the protocol and the server shows as failed in `/mcp`. Log to stderr.
- If `/mcp` shows the server failed, run the exact command from `.mcp.json` yourself in a terminal: the traceback is usually a wrong Python path or running from the wrong folder.
- Memory is data too: a reviewer note could contain PHI or an injection. You masked it; in production you would also treat it as untrusted input to the agent.

Stretch goal

Turn the Claude Code side into an agent too: create a project subagent in `.claude/agents/discharge-reviewer.md` whose tools are limited to the dischargeflow MCP tools and Read, with instructions to pre-screen pending drafts against SPEC AI-03 and AI-04 and report concerns. Ask Claude Code to use it on the review queue. Your reviewers now have an assistant that can check, but never sign.